弦域·织音 技术报告

技术报告 | 2026 年 11 月

摘要

弦域·织音将真实键盘演奏、FPGA 实时合成、曲谱管理与可视化组合成一套电子乐器。USB-MIDI 电钢琴提供音符、力度、延音踏板和音色事件；ESP32-S3 完成 USB Host 接入、SPI/READY 实时事件转发及 BLE 曲谱传输，摇杆、旋钮和双按键提供连续弯音、颤音深度与八度控制。Tang Mega 60K 中的 HDL 完成四声部分配、相位推进、包络、钢琴 / 风琴 / 加法音色、混音、回声和双路数字音频输出。钢琴支路为四个声部保存各自的分量相位，每声部最多计算 32 个加法分量，通过频率拉伸、分量权重、音头动态和带外分量抑制塑造钢琴音色。50 MHz 时钟每 1024 拍形成一个采样周期，对应 48,828.125 Hz；PCM5102A 和板载 PT8211 可输出同一实时生成的声音。HDMI 将波形、FFT 频谱、键盘反馈、五线谱、简谱和钢琴块练习连接到演奏过程。配套网站提供曲谱搜索、筛选、预览、导入与蓝牙下载，FPGA 从 microSD 读取 PSC2 曲谱用于显示和练习。2026-10-24 的布局布线报告显示系统域 Fmax 50.028 MHz、像素域 Fmax 25.811 MHz，setup/hold 违例端点均为 0。作品的核心是让 FPGA 的确定性逐样本运算直接服务于可演奏的音色和交互。

第一部分 作品概述

1.1 功能与特性

演奏者使用电钢琴完成选音、力度、踏板和音色切换，并以摇杆和旋钮连续塑造音高与颤音。ESP32-S3 将 USB-MIDI 事件经 READY 握手送给 FPGA。FPGA 在四个独立声部中分配音符，实时生成 P 钢琴、O 风琴和 A 加法音色；支持 ADSR、弯音、颤音、四声部混音及可开关回声。数字音频同时进入板载 PT8211 和外置 PCM5102A 标准 I²S 声路。HDMI 显示演奏波形、64 点 FFT 频谱、琴键反馈、四频测试、五线谱、简谱和钢琴块练习，练习画面叠加起音判定、连击与计数。网站完成曲库浏览、曲谱预览、.pseq 导入、设备目录查询和蓝牙下载。

1.2 应用领域

适用于可编程电子乐器教学、FPGA 实时数字信号处理实验、交互式音频系统验证和曲谱辅助练习。USB 电钢琴提供熟悉的演奏界面，摇杆、旋钮与按键增加连续表情；本地曲谱和练习反馈使合成引擎兼具演奏与学习用途。

1.3 主要技术特点

四声部各有独立音符、频率 / 相位和包络状态。钢琴支路采用 128 份分量相位状态，以共享正弦查表和乘法流水线在 1024 拍内完成四声部、每声部至多 32 分量的计算；高音依 Nyquist 界限缩减分量。包络和击键力度共同塑造发音动态。16 位有符号混音采样装入左右各 32 位的 I²S 槽，送至 PCM5102A；同一混音并行送至 PT8211。64 点 FFT 与 HDMI 页用于现场观察声音响应。



图 1 音频采样周期与分量计算预算（设计示意，按 RTL 时钟推导；不是示波器实测）。

1.4 主要性能指标

指标	当前证据与边界
演奏声部	4 个独立声部，含力度、包络和踏板状态。
加法分量	P 每声部至多 32 分量；高音受 Nyquist 限制。
采样率	50 MHz / 1024 = 48,828.125 Hz（RTL 推导）。
I²S	BCK 3.125 MHz、LRCK 48.828125 kHz（推导）。
时序	10 月 24 日 PnR：系统域 50.028 MHz，像素域 25.811 MHz，setup/hold 0/0。
输出链路	PCM5102A 标准 I²S 与板载 PT8211 并行。

1.5 主要创新点

1. 四个声部各自保存相位、频率、包络和击键状态；钢琴支路再为每个声部保存最多 32 路分量相位，以采样周期内的硬件调度实现细致的频谱塑形。
2. 音色计算、实时效果、FFT 与 HDMI 共处一套 FPGA 位流，演奏者听到的声音与看到的波形、频谱和键盘状态由同一实时事件驱动。
3. 曲谱从网页经 BLE、UART 和 microSD 进入 FPGA，复用为五线谱、简谱及钢琴块练习数据，构成演奏、观察与学习的连续体验。
4. 标准 I²S 的 PCM5102A 与板载 PT8211 并行接收同一混音采样，使音源算法与数字音频接口能够分别展示。

1.6 设计流程

先建立 MIDI 事件与四声部音源，再接入 USB 电钢琴、SPI/READY 和音频输出；随后加入摇杆表情、32 分量钢琴、回声与 HDMI 可视化。曲谱经网站打包、BLE-UART 代理、FPGA microSD 存储和 PAGE/BITMAP 显示进入系统；最终接入标准 I²S DAC，并用仿真、布局布线报告和实机演奏交叉检查。

第二部分 系统组成及功能说明

2.1 整体介绍



图 2 系统架构与模块连接关系。

ESP32-S3 是控制与文件传输桥，不参与采样值计算；FPGA 是完整音源。microSD 的曲目数据和 HDMI 曲谱阅读属于附加系统功能，不是预录 PCM 音频播放。所有声学输出均从实时波形算法产生。



图 3 从 MIDI 事件到 PCM5102A 模拟输出的数据通路（设计示意）。

2.2 硬件系统介绍

2.2.1 硬件整体介绍

核心板为 Tang Mega 60K；ESP32-S3 USB Host 接入电钢琴，GPIO12/11/10 分别提供 SPI SCLK/MOSI/CS，GPIO13 接收 FPGA READY，双方共用 3.3 V 逻辑及公共地。曲谱全双工 UART1 用 GPIO17/18。摇杆 X1→GPIO1 为回中弯音，Y1→GPIO2 为瞬时颤音深度微调，RP1→GPIO4 为保持型深度旋钮；SW1→GPIO5、SW2→GPIO6 分别逐次降低 / 升高八度，范围 -4 至 +4。模拟输入只接 3.3 V 供电的摇杆，不能输入 5 V。

外置 PCM5102A 从 NEO Dock J8/PMOD1 接收 BCK、LRCK、DIN，VIN 接 5 V 电源并与 FPGA 共地；LINE OUT 接有源音箱。板载 PT8211 同步输出，构成两条可对照的数字音频接口。

2.2.2 机械设计介绍

样机以电钢琴作为完整力度键盘，Tang Mega 60K、ESP32-S3、摇杆、DAC 与有源音箱布置在演奏区周围。分体式构造使输入控制、合成核心、曲谱存储和音频输出的信号流清晰可见，也便于现场切换音色、页面和演奏模式。

2.2.3 电路各模块介绍

USB 电钢琴接 ESP32-S3 的 USB Host；GPIO12/11/10 为 SPI SCLK/MOSI/CS，GPIO13 接 FPGA READY，GPIO17/18 为曲谱 UART。FPGA 的 HP_BCK/WS/DIN 连接板载 PT8211，PCM_BCK/WS/DIN 连接外置 PCM5102A；HDMI 输出画面，SD SPI 接口访问 microSD。摇杆 ADC 采用 3.3 V 电平。top.v 将上述接口明确分为演奏事件、曲谱传输、音频和视频四组，信号关系可由顶层端口逐一对应到实体模块。

2.3 软件系统介绍

2.3.1 软件整体介绍

FPGA 入口 src/top.v 连接 spi_midi_frame_rx.v、poly_voice_ctrl.v、basic_dds_synth.v、piano32_poly_synth.v、additive32_synth.v、echo_delay.v 和 pcm5102_i2s_tx.v。FPGA 的曲谱通路由 UART v2 控制器、SD 块设备、PSC2 PAGE 索引与缓存组成；视频通路由 640×480 时序、波形/64 点 FFT、五线谱/简谱页面与钢琴块练习叠加组成。src/timing.sdc 给 sys_clk 设置 20 ns 约束，并为显示 PLL 建立像素域时钟。

ESP-IDF 工程 esp32_local/main/ 中，USB Host 解析 USB-MIDI 1.0 包，桥接器按 READY 状态排队发送 SPI 帧；Program Change 透传到 FPGA，摇杆产生 Pitch Bend 和 CC1，按键通过 MIDI 音高映射实现八度调整。曲谱由团队网站提供：曲库页展示《天真》《进步》《阿拉贝斯克》的封面谱图、时长、节拍、音符数和文件大小，可搜索、筛选、预览与导入 .pseq 曲目包；设备页通过 Web Bluetooth 连接 ESP32-S3，查询板上目录并发送曲谱。网站 Library.vue 负责曲库搜索、筛选、卡片及预览交互，useDevice.ts 编排上传、续传与设备状态，ble.ts 实现蓝牙传输，psc2.ts 处理 PAGE/NOTE 等数据段及 CRC。曲目卡片同时标明五线谱、简谱和钢琴块三种呈现方式；左侧“发现曲谱 / 我的设备 / 使用指南”将选曲、设备管理和操作说明放在同一入口。网页源码由团队独立仓库维护。

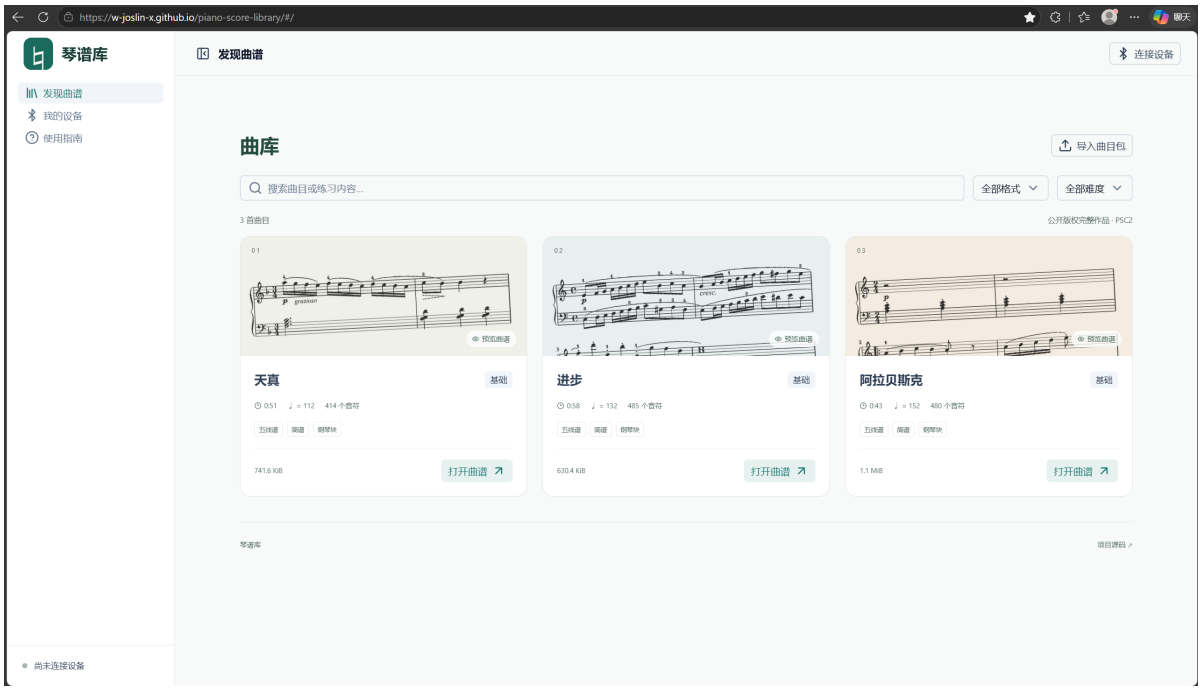


图 4 网站曲库页面：谱图预览、曲目信息、三种呈现方式、筛选及导入入口。

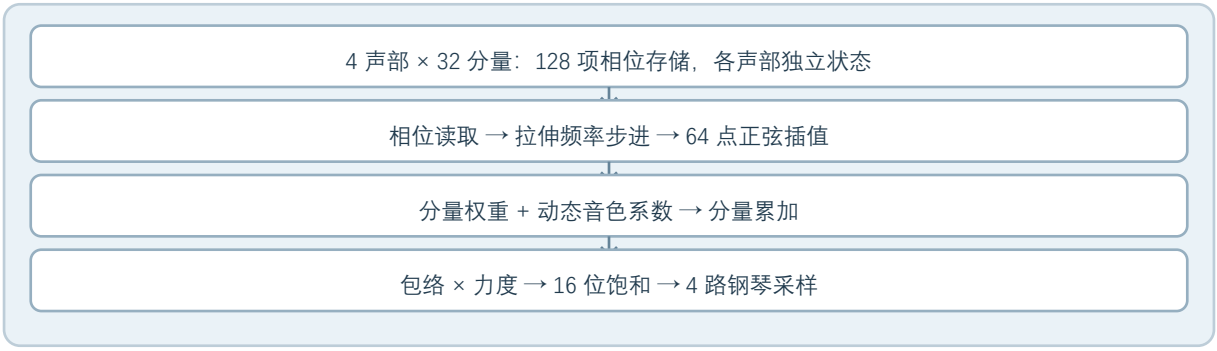
2.3.2 软件各模块介绍

输入与分配。 spi_midi_frame_rx.v 将 SPI 串行帧还原为事件类型、通道、音符与力度；poly_voice_ctrl.v 为四个演奏声部优先分配空闲槽，重复 Note On 复用原声部，延音踏板 CC64 使松键后的音继续保持，满额时按轮转策略换声部。top.v 以 50 MHz 系统时钟每 1024 拍产生一次 sample_tick，所有音频采样逻辑共享该节拍。basic_dds_synth.v 依据 MIDI 音高生成相位步进，弯音因子乘法分级寄存；每个声部维护相位、力度、ADSR 状态。风琴预置使用正弦谐波叠加与持续包络。

钢琴音色。 piano32_poly_synth.v 的 128 项 phase_mem 对应四声部各 32 个分量。每次 sample_tick 锁存四声部的频率、包络、力度与音符号，状态机按声部和分量次序执行相位读取、频率系数计算、正弦插值、权重修正和累加。分量频率使用带轻微拉伸的 harmonic_multiplier()；partial_weight() 设置各分量强度，音头与衰减中的高频贡献再由包络调制。max_partial() 随音高限制最高分量，控制带外成分。分量和经包络、力度缩放与 16 位饱和，得到四路独立钢琴采样。这是一条完整的周期内共享运算路径：音色计算的层次来自 32 分量，复音来自四份独立声部状态。

钢琴合成的关键并非播放一段固定波形，而是在每个采样点重新计算。sample_tick 到来时，step_bus、envelope_bus、velocity_bus 和 note_bus 同时锁存，避免同一采样点内各分量使用不同控制值。{voice_index, partial_address} 形成相位存储地址，因此相同编号的分量在四个声部中仍有独立的相位轨迹。PHASE_READ 读出旧相位，PHASE_CALC 用基础步进乘以预设的分量频率系数；重触发时使用 phase_offset() 建立音头相位关系。SINE 阶段对 64 点正弦表做插值，并写回下一采样点的相位；后续 TONE、WEIGHT、ACCUM 将正弦值依次乘以静态频谱权重和随包络变化的音色系数，再累加进当前声部。LEVEL、VELOCITY、OUTPUT 依次完成包络、力度和饱和输出。四个声部重复同一套系数和计算规则，因此合成音色保持一致。

音色参数体现了对钢琴声音结构的具体处理。harmonic_multiplier() 让较高分量的频率稍高于严格整数倍，模拟琴弦非理想刚性带来的谱峰拉伸；partial_weight() 用 32 组系数安排基波与高阶分量的相对强度，其中第 6、9 分量得到局部增强；tone_factor 让高频分量随音头和衰减改变贡献，低阶分量则保持主体音高。max_partial() 根据 MIDI 音高降低高音区的最高分量，避免分量越过采样率一半后折叠回可闻频带。128 份相位状态与共享运算链的组合，是本作品在音色层次、四声部独立性和 FPGA 资源之间的核心设计。



钢琴合成计算图：四声部相位记忆、共享分量运算、包络力度与四路输出。

piano32_poly_synth.v lines 16-31

```
16      localparam [3:0] IDLE=0, LOAD=1, PHASE_READ=2, PHASE_CALC=3,
17                       SINE=4, TONE=5, WEIGHT=6, ACCUM=7,
18                       LEVEL=8, VELOCITY=9, OUTPUT=10;
19      reg [3:0] state;
20      reg [1:0] voice_index;
21      reg [5:0] partial, partial_limit;
22      reg [127:0] step_snapshot;
23      reg [31:0] envelope_snapshot;
24      reg [27:0] velocity_snapshot, note_snapshot;
25      reg [31:0] step_current, phase_h_q, step_h_q;
26      reg [3:0] reset_pending, reset_snapshot;
27      reg previous_enable;
28      reg [31:0] phase_mem [0:127];
29      reg [31:0] phase_mem_rdata;
30      wire [4:0] partial_address = partial[4:0] - 5'd1;
31      wire [6:0] memory_address = {voice_index, partial_address};
```

代码框 A：状态机、声部索引与 128 项分量相位存储。

piano32_poly_synth.v lines 326-354

```
326                     PHASE_CALC: begin
327                         phase_h_q<=reset_snapshot[voice_index] ?
328                                   phase_offset(partial) : phase_mem_rdata;
329                         step_h_q<=step_product[47:16];
330                         weight_base_q<=partial_weight(partial);
331                         state<=SINE;
332                     end
333                     SINE: begin
334                         sine_q<=sine_interpolated(phase_h_q);
335                         if (partial<=6'd4) tone_factor<=8'd255;
336                         else if (partial==6'd6 || partial==6'd9)
337                             tone_factor<=8'd192+(envelope_current>>2);
338                         else if (partial>=6'd13) tone_factor<=envelope_current;
339                         else tone_factor<=8'd63+envelope_current-(envelope_current>>2);
340                         state<=TONE;
341                     end
342                     TONE: begin
343                         weight_q<=tone_product[23:8];
344                         state<=WEIGHT;
345                     end
346                     WEIGHT: begin
347                         weighted_q<=harmonic_product >>> 15;
348                         state<=ACCUM;
349                     end
350                     ACCUM: begin
351                         sum_q<=sum_q+weighted_q;
352                         if (partial==partial_limit) state<=LEVEL;
353                         else begin partial<=partial+1'b1; state<=PHASE_READ; end
354                     end
```

代码框 B：分量计算从相位、正弦插值到加权累加的关键状态。

piano32_poly_synth.v lines 35-54

```
35      reg signed [16:0] sine_q;
36      reg signed [17:0] weighted_q;
37      reg signed [31:0] sum_q;
38      reg signed [39:0] level_product_q;
39      reg signed [46:0] velocity_product_q;
40      wire [53:0] step_product = step_current * harmonic_multiplier(partial);
41      wire [23:0] tone_product = weight_base_q * tone_factor;
42      wire signed [33:0] harmonic_product =
43          $signed(sine_q) * $signed({1'b0, weight_q});
44      wire signed [46:0] scaled = velocity_product_q >>> 15;
45      wire signed [15:0] saturated =
46          (scaled > 47'sd32767) ? 16'sh7fff :
47          (scaled < -47'sd32768) ? 16'sh8000 : scaled[15:0];
48
49      // Synchronous single-port memory: 128 independent partial phases. An
50      // inharmonic oscillator must keep its own phase; multiplying the wrapped
51      // fundamental phase would introduce a discontinuity every base period.
52      always @(posedge clk) begin
53          if (state == PHASE_READ) phase_mem_rdata <= phase_mem[memory_address];
54          if (state == SINE) phase_mem[memory_address] <= phase_h_q + step_h_q;
```

代码框 C：分量频率系数乘法、正弦 / 权重乘法和相位存储的共享硬件路径。

基础 DDS、包络与风琴。 basic_dds_synth.v 在每个演奏声部内保存 32 位相位和独立包络状态，MIDI 音高决定基础相位步进，力度与包络决定输出幅度。正弦波来自 64 点查表及相邻采样插值；风琴将多个整数倍谐波合成，保持更稳定的持续段，并加入短促的触键瞬态。top.v 共享弯音系数给四个声部：14 位 Pitch Bend 经分段查表和插值转为频率因子，CC1 驱动三角波颤音并与弯音叠加，控制变化时维持声部相位连续。poly_voice_ctrl.v 处理 Note On、Note Off、重触发、满额换声部和 CC64 延音踏板，使演奏事件转成四路稳定的发声状态。

加法音色、混音与回声。 A 音色由 additive32_synth.v 在一个采样周期内对 32 个分量逐项求和。top.v 按 Program Change 在钢琴、风琴、加法分支之间选择，同一组键盘事件可直接对比三种音色。四路采样先符号扩展求和，再算术右移两位给满额和弦留出幅度余量。echo_delay.v 使用 8192×16 位同步延时线保存历史采样，按开关将延时声叠加到干声；audio_effected 同时送到两路 DAC 和 FFT，因此听觉输出与频谱显示来自相同的效果后采样。

top.v lines 225-242

```
225                         spectral_active ? spectral_sample : voice0_basic_sample;
226     assign voice1_sample = piano32_active ? piano32_samples[31:16] : voice1_basic_sample;
227     assign voice2_sample = piano32_active ? piano32_samples[47:32] : voice2_basic_sample;
228     assign voice3_sample = piano32_active ? piano32_samples[63:48] : voice3_basic_sample;
229     wire signed [17:0] mix_sum =
230         {{2{voice0_sample[15]}}, voice0_sample} +
231         {{2{voice1_sample[15]}}, voice1_sample} +
232         {{2{voice2_sample[15]}}, voice2_sample} +
233         {{2{voice3_sample[15]}}, voice3_sample};
234     // Divide by four so four full-scale voices cannot clip the 16-bit DAC.
235     wire signed [15:0] audio_sample = mix_sum >>> 2;
236     wire signed [15:0] audio_effected;
237     echo_delay u_echo_delay (
238         .clk(sys_clk), .rst_n(voice_rst_n),
239         .sample_tick(sample_tick),
240         .enable(echo_enabled && !spectrum_test_active),
241         .dry_in(audio_sample), .audio_out(audio_effected)
242     );
```

代码框 D：四声部求和、幅度预留和回声插入位置。

音频接口。 pt8211_tx.v 将混音采样发送至板载声路；pcm5102_i2s_tx.v 独立产生标准 I²S 的 BCK、WS 与 DIN，左右声道均使用同一 16 位混音采样并填入 32 位时隙。两个发送器由同一 audio_effected 驱动，使数字音源与 DAC 接口能够并行核对。50 MHz / 16 形成 3.125 MHz BCK，64 个 BCK 构成一组左右采样帧，对应 48.828125 kHz。

top.v lines 497-509

```
497     pt8211_tx u_dac_tx (
498         .clk(sys_clk), .rst_n(core_rst_n),
499         .sample_left(audio_effected), .sample_right(audio_effected),
500         .bck(HP_BCK), .ws(HP_WS), .din(HP_DIN)
501     );
502
503     // Independent standard-I2S output on J8/PMOD1 for an external PCM5102A.
504     // Keep the PT8211 path and PA_EN unchanged for immediate A/B comparison.
505     pcm5102_i2s_tx u_pcm5102_tx (
506         .clk(sys_clk), .rst_n(core_rst_n),
507         .sample_left(audio_effected), .sample_right(audio_effected),
508         .bck(PCM_BCK), .ws(PCM_WS), .din(PCM_DIN)
509     );
```

代码框 E：板载 PT8211 与外置 PCM5102A 两条发送通路共用一个音频采样。

频谱与 HDMI。 fft64_spectrum.v 对混音采样先做 16 点平均抽取，再采集 64 点并以六级基 2 FFT 计算；一套复数蝶形单元依次完成各级运算，输出 32 个幅值给频谱柱。midi_visualizer.v 显示演奏波形、频谱、音色标记和琴键反馈；波形区选择当前较响的声部，频谱区使用完整混音，因此和弦的多音关系仍能在频谱中看到。video_timing_640x480.v 生成像素坐标和同步时序，video_pll.v 产生像素与串行时钟，dvi_tmds_encoder.v 和 dvi_tmds_phy.v 将 RGB 像素送至 HDMI。spectrum_test_pattern.v 生成四个不同音高的测试事件，用于现场展示并发谱峰。

曲谱与练习。 score_uart_v2_transport.v / score_store_v2.v 实现网站到板上的曲谱协议、目录和存储控制，sd_spi_block_device.v 以块读写访问 microSD。psc2_page_index.v 定位曲谱中的 PAGE/BITMAP 数据，score_page_cache.v 以 1 bit/像素将 640×480 谱页放进 38,400 字节缓存，score_page_video.v 将缓存字节映射成五线谱或简谱画面。view_key_control.v 识别单击切谱式、双击翻页、长按退出，页面请求和完成信号保证显示的是同一页数据。psc2_note_stream.v 与 piano_preview_cache.v 从曲目音符流生成钢琴块参考数据，practice_onset_match.v 将演奏起音和参考音符匹配，practice_onset_stats.v 汇总判定及连击，practice_stats_video.v、practice_combo_video.v 叠加到画面。网站预览、板上两类谱页和练习数据因此围绕同一份 PSC2 曲目组织。

FPGA 功能可按输入到输出的路径逐一对应到源码：

功能层	RTL 模块与作用
演奏接入	spi_midi_frame_rx.v 接收 SPI/READY 帧；midi_frame_rx.v 与 uart_rx.v 保留串口输入；spectrum_test_pattern.v 生成测试事件。
声部分配	poly_voice_ctrl.v 管理四声部、重触发和延音；top.v 分配 Program Change、Pitch Bend 与 CC1。
声音生成	basic_dds_synth.v 形成 DDS、包络和风琴；piano32_poly_synth.v 形成四声部钢琴分量；additive32_synth.v 形成 A 音色。
效果与输出	top.v 混音；echo_delay.v 产生回声；pt8211_tx.v 和 pcm5102_i2s_tx.v 发送两路数字音频。

功能层	RTL 模块与作用
实时可视化	fft64_spectrum.v 算 64 点 FFT；midi_visualizer.v 绘波形、频谱、音色与琴键；视频时序、TMDS 编码和 PHY 连接 HDMI。
曲谱存储	score_uart_v2_transport.v、score_store_v2.v 完成传输、目录及请求；sd_spi_block_device.v 和 sd_directory_* 模块访问卡与目录。
曲谱显示	psc2_page_index.v、sd_score_page_read.v、score_page_cache.v、score_page_video.v 完成定位、读取、缓存与逐像素显示。
练习反馈	psc2_note_stream.v、piano_preview_cache.v、piano_blocks_demo.v 形成参考块；practice_onset_match.v、practice_onset_stats.v 形成判定与统计。
模式与时钟	view_key_control.v 和 spectrum_mode_key.v 解析按键；video_pll.v、video_timing_640x480.v 建立显示时序。

top.v lines 62-63

```
  62      reg [9:0] sample_counter;
  63      wire sample_tick = (sample_counter == 10'd1023);
```

代码框 F：1024 拍采样使能，来自当前 FPGA 源码。

top.v lines 210-223

```
 210      piano32_poly_synth u_piano32_poly_synth (
 211          .clk(sys_clk), .rst_n(voice_rst_n), .sample_tick(sample_tick),
 212          .enable(piano32_active),
 213          .step_bus({step3, step2, step1, step0}),
 214          .retrigger_bus(retrigger),
 215          .envelope_bus({envelope3, envelope2, envelope1, envelope0}),
 216          .velocity_bus({velocity3, velocity2, velocity1, velocity0}),
 217          .note_bus({note3, note2, note1, note0}),
 218          .sample0(piano32_samples[15:0]),
 219          .sample1(piano32_samples[31:16]),
 220          .sample2(piano32_samples[47:32]),
 221          .sample3(piano32_samples[63:48]),
 222          .sample_valid(piano32_sample_valid), .overrun(piano32_overrun)
 223      );
```

代码框 G：四声部钢琴音色的输入与四路采样输出，来自当前 FPGA 源码。

top.v lines 505-511

```
 505      pcm5102_i2s_tx u_pcm5102_tx (
 506          .clk(sys_clk), .rst_n(core_rst_n),
 507          .sample_left(audio_effected), .sample_right(audio_effected),
 508          .bck(PCM_BCK), .ws(PCM_WS), .din(PCM_DIN)
 509      );
 510
 511      // NEO Dock schematic: PA_EN=0 means amplifier on.
```

代码框 H：与 PT8211 并行的 PCM5102A 标准 I²S 输出，来自当前 FPGA 源码。



图 5 HDMI 页面关系（设计示意；实际页面照片见第三部分）。

第三部分 完成情况及性能参数

3.1 整体介绍

样机由电钢琴、Tang Mega 60K、ESP32-S3、摇杆控制区、PCM5102A、显示器与有源音箱构成。电钢琴提供熟悉的演奏动作，摇杆补充连续表情，FPGA 负责全部音频样本计算；网页、microSD 和 HDMI 将曲目数据带入演奏过程。各模块围绕琴键布置，形成可直接演奏、可观察合成过程的工作台。



图 6 样机与各硬件模块的布置。

3.2 工程成果

3.2.1 机械成果

电钢琴提供完整键盘尺度，FPGA、ESP32、DAC 与摇杆控制区布置在演奏者触手可及的位置。模块间采用可见连接，使电钢琴输入、硬件合成、音频转换和 HDMI 显示之间的关系一目了然。

3.2.2 电路成果

电路链路从电钢琴 USB 输入开始，经 ESP32-S3 的 SPI/READY 桥进入 FPGA；FPGA 同时连接 PT8211、PCM5102A、HDMI 与 microSD。top.v 的 spi_*、HP_BCK/WS/DIN、PCM_BCK/WS/DIN 和视频端口分别对应这些实体接口。外置 DAC 使用标准 I²S，板载 PT8211 声路保留并行输出，便于音频对照。

3.2.3 软件成果

HDMI 演奏页以 P/O/A 标记当前钢琴、风琴或加法音色，上半部显示主声部的时域波形，下半部显示完整混音的 FFT 频谱，底部琴键图形跟随演奏事件。演奏者可边听音色、边观察波形与频谱形状。图 7 展示页面布局。

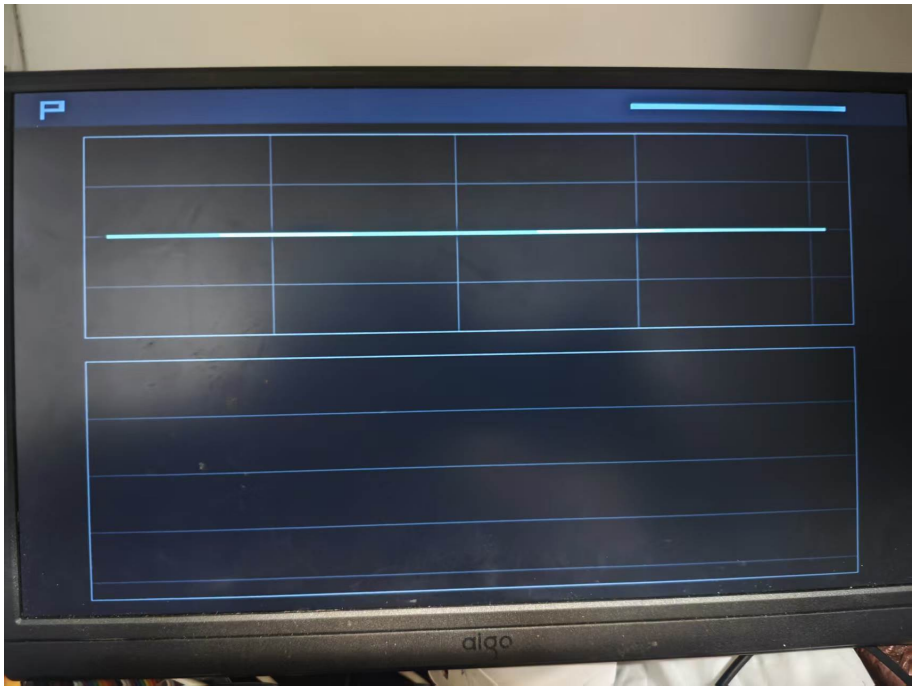


图 7 HDMI 演奏页的波形、频谱与琴键布局。

练习页显示落下的钢琴块、键盘图形以及顶部 P/G/M/X 计数。音符流和演奏起音进入匹配逻辑，判定结果与连击统计由 HDL 绘制到画面顶部。

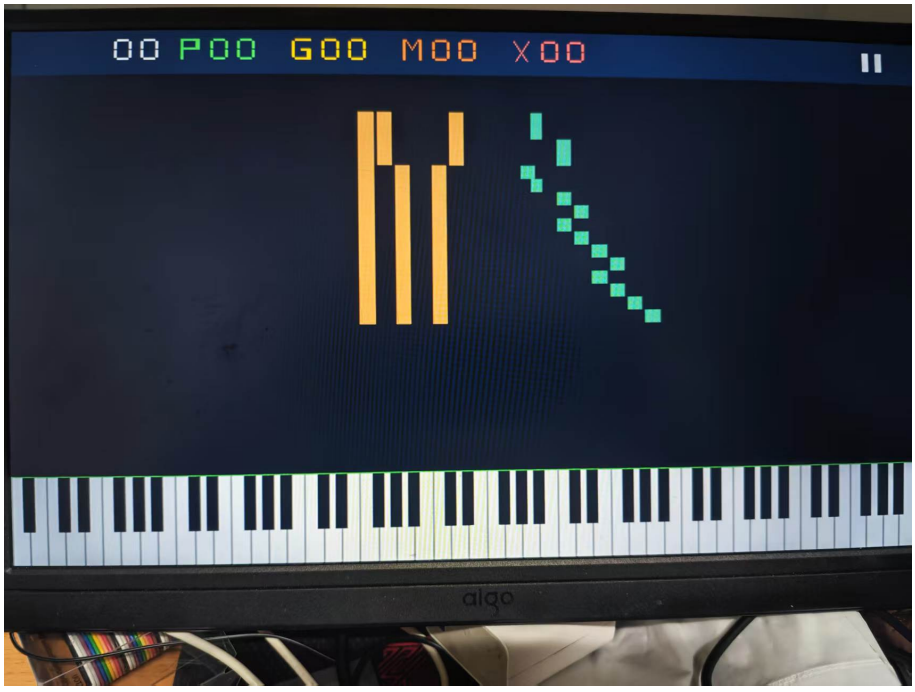


图 8 钢琴块练习页（实机照片）。

五线谱与简谱照片展示 PSC2 PAGE/BITMAP 经 microSD 读取、片上缓存和 HDMI 像素输出的结果。两种谱式都由同一曲目文件提供，并可通过按键切换。

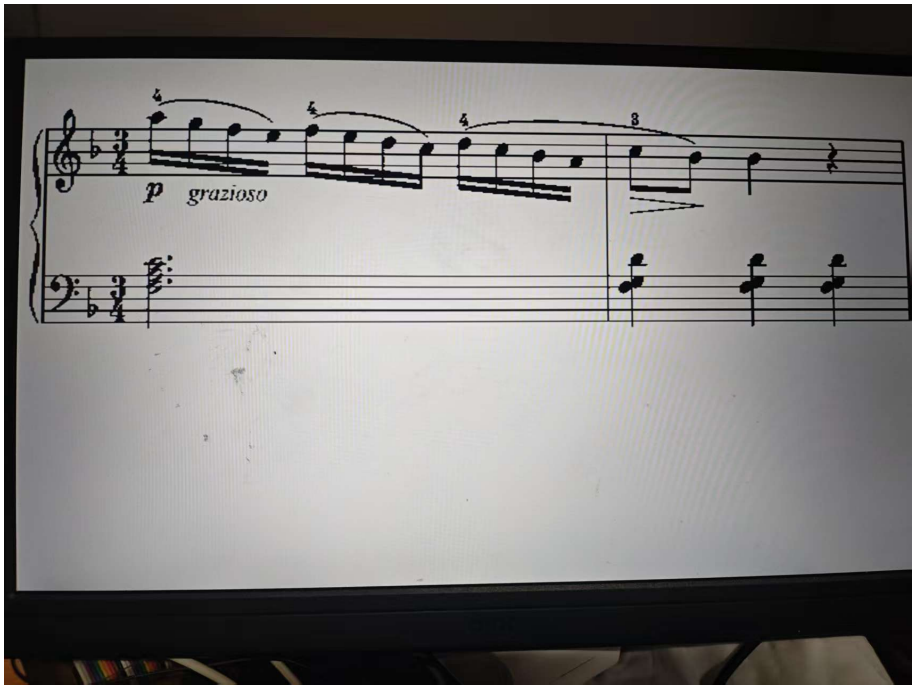


图 9 五线谱页（实机照片）。

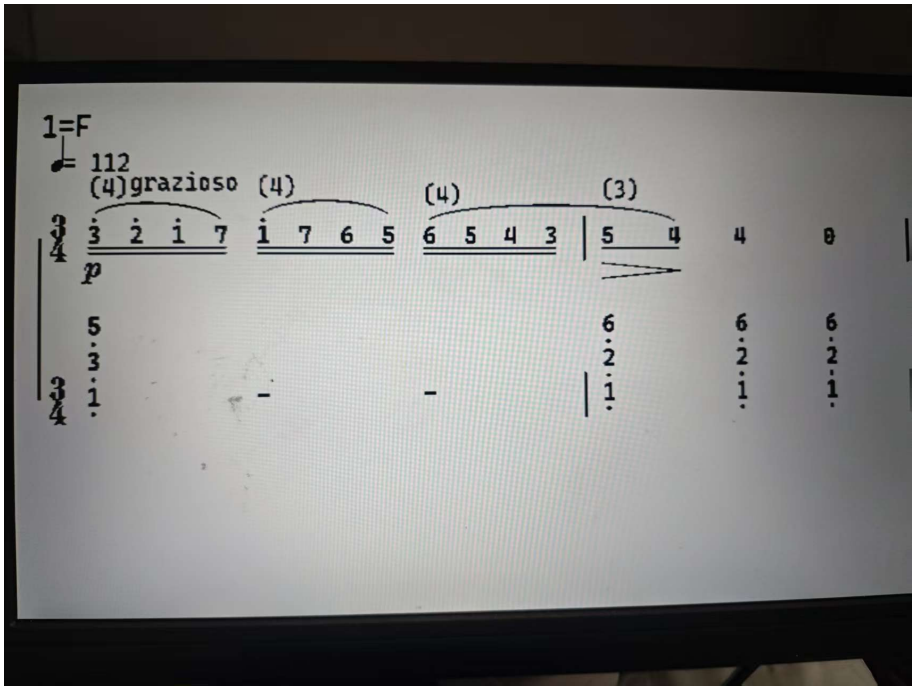
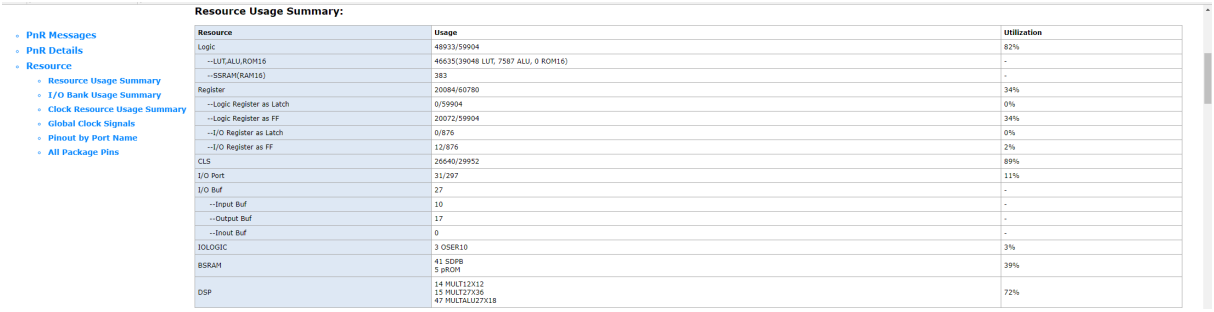


图 10 简谱页（实机照片）。

3.3 特性成果

FPGA top.v 以 50 MHz 时钟和 1024 拍计数形成采样使能，设计采样率为 48,828.125 Hz。poly_voice_ctrl.v 管理四个演奏声部，piano32_poly_synth.v 为每声部在周期内计算至多 32 个加法分量，pcm5102_i2s_tx.v 输出标准 I²S。四音测试模式生成 MIDI 55、62、69、76 的不同基频；HDMI 的 64 点 FFT 对完整混音做实时频谱反馈。作品同时具备细致的钢琴音色、可演奏的多声部、标准数字音频接口和现场可视化。

2026-10-24 19:00 的布局布线报告给出 Logic 48,933/59,904（82%）、CLS 26,640/29,952（89%）、BSRAM 39%、DSP 72%。这一资源分布反映了四声部加法钢琴、曲谱存储与视频显示并存的硬件规模。



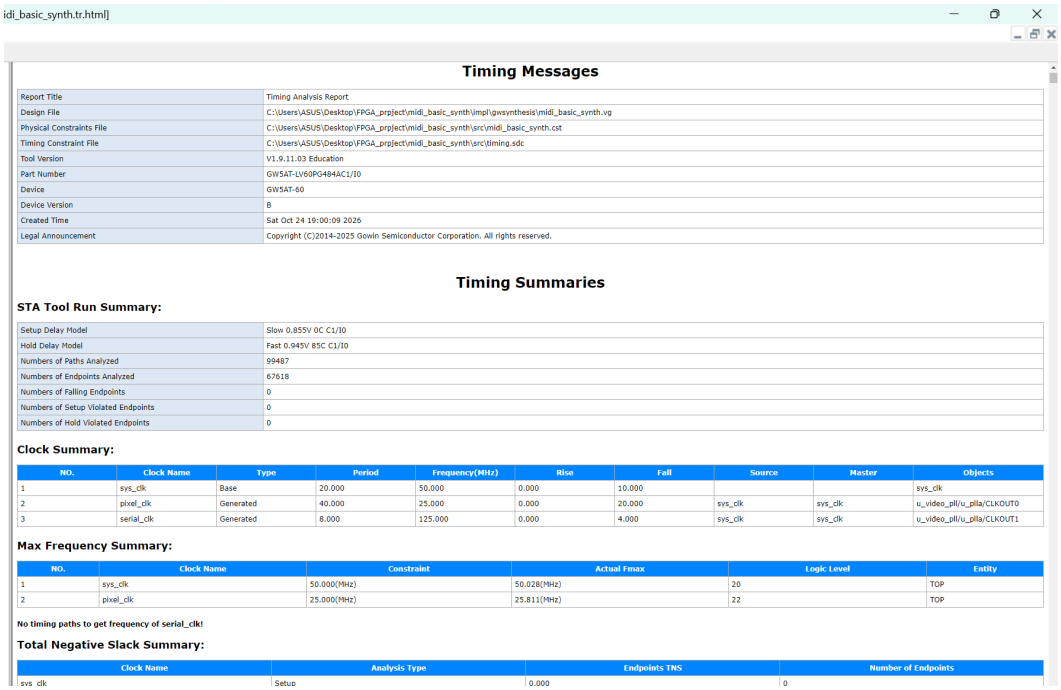
- PnR Messages
- PnR Details
- Resource
  - Resource Usage Summary
  - I/O Bank Usage Summary
  - Clock Resource Usage Summary
  - Global Clock Signals
  - Pinout by Port Name
  - All Package Pins

Resource Usage Summary:

Resource	Usage	Utilization
Logic	48933/59904	82%
--LUT,ALU,ROM16	46635(39048 LUT, 7587 ALU, 0 ROM16)	-
--SSRAM(RAM16)	383	-
Register	20084/60780	34%
--Logic Register as Latch	0/59904	0%
--Logic Register as FF	20072/59904	34%
--I/O Register as Latch	0/876	0%
--I/O Register as FF	12/876	2%
CLS	26640/29952	89%
I/O Port	31/297	11%
I/O Buf	27	-
--Input Buf	10	-
--Output Buf	17	-
--Inout Buf	0	-
IOLOGIC	3 OSER10	3%
BSRAM	41 SDPB 5 pROM	39%
DSP	14 MULT12X12 15 MULT27X36 47 MULTALU27X18	72%

图 11 2026-10-24 布局布线资源占用截图。

同次构建的完整时序截图给出 sys_clk 约束 50 MHz、实际 Fmax 50.028 MHz，pixel_clk 约束 25 MHz、实际 Fmax 25.811 MHz；分析 99,487 条路径、67,618 个端点，setup/hold 违例端点均为 0。这表明该版设计满足所设的两个工作时钟约束。



idi_basic_synth.tr.html]

Timing Messages

Report Title	Timing Analysis Report
Design File	C:\Users\ASUS\Desktop\FPGA_prpject\midi_basic_synth\impl\gwsynthesis\midi_basic_synth.vg
Physical Constraints File	C:\Users\ASUS\Desktop\FPGA_prpject\midi_basic_synth\src\midi_basic_synth.cst
Timing Constraint File	C:\Users\ASUS\Desktop\FPGA_prpject\midi_basic_synth\src\timing.sdc
Tool Version	V1.9.11.03 Education
Part Number	GW5AT-LV60PG484AC1/I0
Device	GW5AT-60
Device Version	B
Created Time	Sat Oct 24 19:00:09 2026
Legal Announcement	Copyright (C)2014-2025 Gowin Semiconductor Corporation. All rights reserved.

Timing Summaries

STA Tool Run Summary:

Setup Delay Model	Slow 0.855V 0C C1/I0
Hold Delay Model	Fast 0.945V 85C C1/I0
Numbers of Paths Analyzed	99487
Numbers of Endpoints Analyzed	67618
Numbers of Falling Endpoints	0
Numbers of Setup Violated Endpoints	0
Numbers of Hold Violated Endpoints	0

Clock Summary:

NO.	Clock Name	Type	Period	Frequency(MHz)	Rise	Fall	Source	Master	Objects
1	sys_clk	Base	20.000	50.000	0.000	10.000			sys_clk
2	pixel_clk	Generated	40.000	25.000	0.000	20.000	sys_clk	sys_clk	u_video_pll/u_plla/CLKOUT0
3	serial_clk	Generated	8.000	125.000	0.000	4.000	sys_clk	sys_clk	u_video_pll/u_plla/CLKOUT1

Max Frequency Summary:

NO.	Clock Name	Constraint	Actual Fmax	Logic Level	Entity
1	sys_clk	50.000(MHz)	50.028(MHz)	20	TOP
2	pixel_clk	25.000(MHz)	25.811(MHz)	22	TOP

No timing paths to get frequency of serial_clk!

Total Negative Slack Summary:

Clock Name	Analysis Type	Endpoints TNS	Number of Endpoints
sys_clk	Setup	0.000	0

图 12 2026-10-24 完整时序摘要：双时钟域 Fmax 与 setup/hold 端点。

Gowin Analyzer Oscilloscope 片上采样显示 SPI 传输、HP_BCK/HP_WS 时钟活动以及内部音频总线变化。该图直接对应 HDL 的输入帧接收和数字音频发送节点，便于检查事件进入 FPGA 后的信号传播。

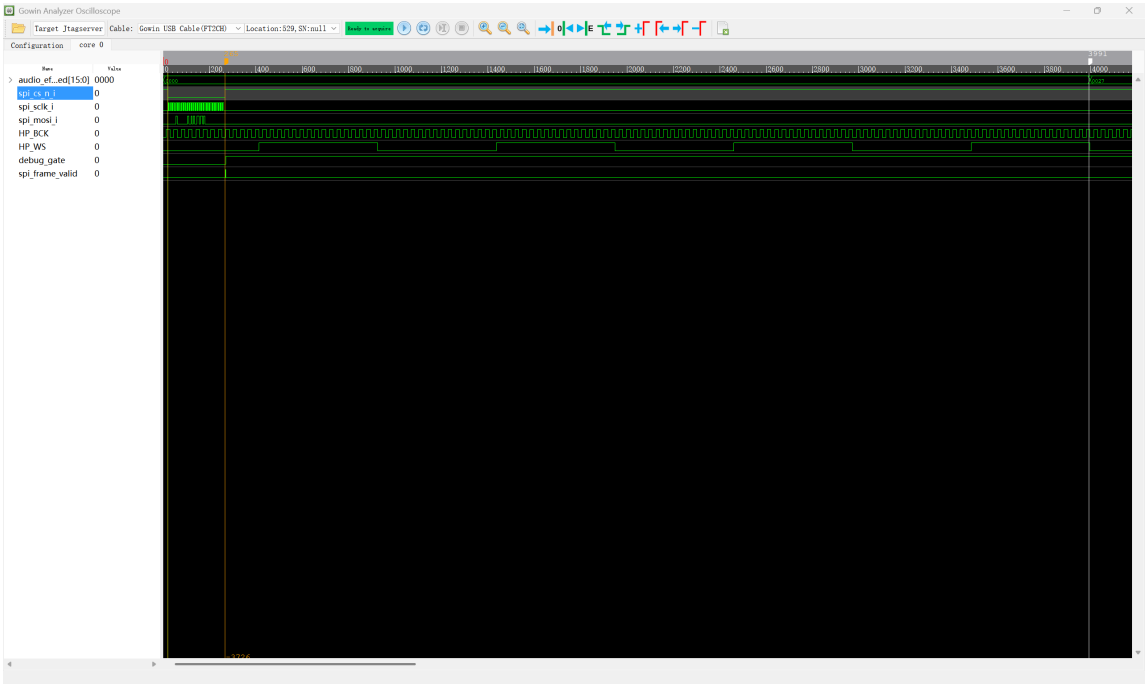


图 13 Gowin 片上采样的 SPI 与数字音频节点。

音频通路的 sim/audio_latency.vcd 与仿真向量覆盖 SPI 帧、音符分配、钢琴 / 风琴 / 加法音色、采样节拍和 PT8211 串行输出。图 14 给出可复现的完整演奏链路测量接法，将琴键动作与 DAC 线路输出放在同一时间轴上观察。

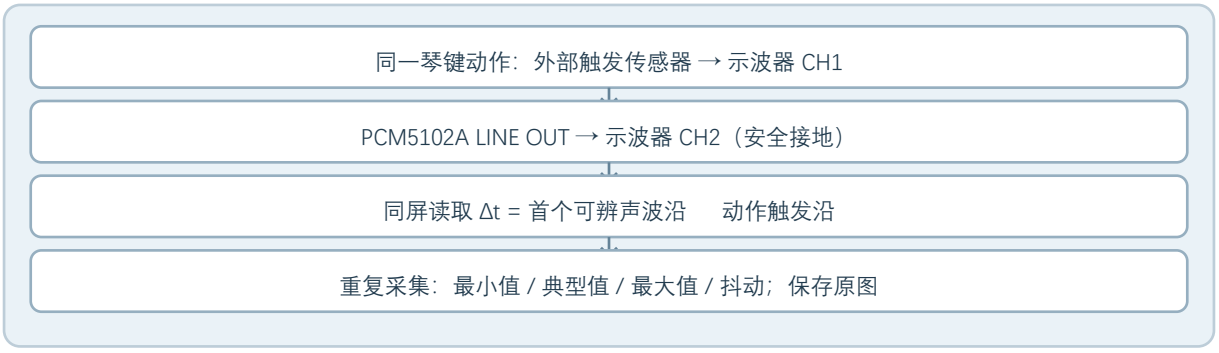


图 14 完整链路延迟的测量接法示意。

第四部分 总结

4.1 可扩展之处

当前架构将演奏事件、声部状态、合成计算和显示存储分成清晰的模块。后续可沿相同接口接入电容触摸音阶、距离控制和光电触发，也可围绕共享分量运算路径增加算术并行度。现有的曲谱协议、页面缓存和起音判定为交互式练习提供了可继续扩展的数据基础。

4.2 心得体会

这套作品的设计重点是把演奏动作直接送进确定性的硬件音频通路。ESP32-S3 承担 USB-MIDI 与 BLE 等连接任务，FPGA 在固定采样周期内完成相位、频谱、包络、效果和音频发送。四声部各自的状态与 32 分量钢琴的共享运算路径相结合，让每个按键既保持独立的发声过程，又获得更丰富的音色层次。PCM5102A 标准 I²S、板载 PT8211、HDMI 频谱和曲谱练习共同把声音算法呈现为可听、可看、可操作的完整系统。

第五部分 参考文献

1. 全国大学生嵌入式芯片与系统设计竞赛，2026 高云半导体赛题文件，选题二，第 15–16 页。
2. Texas Instruments, PCM5100A/PCM5101A/PCM5102A Datasheet，I²S 输入和模拟输出规范。
3. Gowin Semiconductor，GW5AT-60B 器件与 Gowin EDA 时序报告，项目使用的 2026-10-22 19:41 位流构建。
4. USB Implementers Forum，USB Device Class Definition for MIDI Devices 1.0。
5. 本作品 src/、sim/、tests/、esp32_local/main/ 源码和同版本验证记录。